

LangChain: Concepts

Chains:

- * Python class provided by LangChain
- * Usamos Chains para hacer pipelines de generación de texto reutilizables
- * Las Chains se pueden conectar entre sí para hacer un pipeline más complejo
- * Una Chain está compuesta por un "PromptTemplate" y un "LLM"

PromptTemplate:

- * Produces the final prompt that will be sent to the language model (LLM)
- * Debe declarar las variables que necesita para construir el prompt

Language Model (LLM):

* El LLM (ChatGPT, Bard, Gemini, etc) que queremos usar para este pipeline de text generation

Input → Chain [PromptTemplate, LLM] → Output

Input:

- * Dictionary that must contain values for each variable the PromptTemplate requires

Output:

* Dictionary that contain the inputs AND the generated content assigned to the "text" key (The "text" key can be customized)

En LangChain mayormente cuando trabajamos con un LLM este es del tipo "completion" que son los que buscan completar un texto teniendo como contexto lo anterior de ese mismo texto.

(Completion-Style LLM). Eso es lo explicado hasta acá.

Loc conversacionales, como chatGPT, también se pueden usar. Para esto hay que laborar un poquito mas con LangChain. Con estos tengo que distinguir entre los mensajes del bot y los del usuario.

Tipos de mensajes

Existen 3 tipos de mensajes:

OpenAI name | LangChain name

- * User Message | Human Message: el mensaje creado por el usuario
- * Assistant Message | AI Message: El mensaje creado por el modelo de Chat (bot)
- * System Message | System Message: Un mensaje que permite establecer/personalizar como se comporta el chat bot. Generalmente lo sesean los desarrolladores.

ChatGPT no recuerda toda la conversación. Uno debe enviar el historial de mensajes entero cada vez que quiere continuar/extender una conversación.

ChatChain:

* ChatPromptTemplate: Crea la lista de mensajes que le vamos a enviar a ChatGPT. Tiene adentro:

* SystemMessagePromptTemplate

*HumanMessagePromptTemplate

Memorias en LangChain

En una chain, si le pasamos una memoria esta se usa 2 veces.

1. Cuando la cadena recibe la entrada del usuario antes de rellenar la plantilla:
 - tiene la capacidad de agregar variables adicionales antes de crear la plantilla de prompt
2. Al recibir la respuesta del LLM antes de entregar el resultado al programa:
 - después de que el modelo de lenguaje corre, las variables son enviadas a la memoria. La memoria tiene la chance de inspeccionar el resultado y almacenar alguna parte del mismo

Memorias

Existen distintos tipos de memorias:

- ConversationTokenBufferMemory
- CombinedMemory
- ConversationBufferWindowMemory -> Stores only the last K exchanges (un humanMessage y un aiMessage)
- ConversationBufferMemory -> esta sirve para los chat bots (mantener todo el historial de mensajes) -> Stores all messages of the conversation

Para salvar -> En el caso de los chat bots vamos a almacenar en la memoria una list de "mensajes".

Va guardar 2 mensajes -> 1 Human message (la entrada) 2 AI Message (la respuesta)

Para cargar el historial -> va a sesteear en una variable "messages" (o cualquier nombre que le pongamos a memory_key) todo el historial que tiene. si ponemos return_messages = True va a respetar este formato messages = [HumanMessage("cuanto es 1+1?"), AIMessage("3")]

Genial, pero como uso esa variable messages?

Dentro del ChatPromptTemplate:

- MessagesPlaceholder(variable_name = "messages")
- HumanMessagePrompt.from_template = ("{{content}}")

Para salvar este historial de mensajes a la bruto, en un archivo (txt, json, etc) podemos usar FileChatMessageHistory. Obviamente en producción esto no es muy util.

LangChain nos permite usar también SQL, Redes, etc de forma sencilla.

Pero que pasa si el historial crece demasiado? Los LLM tienen un limite de mensajes que pueden recibir y ademas las queries mas grandes son mas caras.

Para eso viene al rescate ConversationSummaryMemory (no nunca bien con

FileChatMessageHistory, se usa 1 o la otra)

ConversationSummaryMemory adentro tiene:

- PromptTemplate: que rellenar con todos los mensajes
- LLM: al que le pide que resuma el historial de mensajes

La diferencia es que en lugar de meter una variable de tipo messages = [HumanMessage("cuanto es 1+1?"), AIMessage("3")], va a insertar un SystemMessage con el texto resumido de todo el historial de mensajes original

LangChain provides classes to help load data from different types of files (json, pdf, txt, md, HTML, css, etc). These classes are called "Loaders". Existen:

- TextLoader for txt
- PyPDFLoader for pdf
- JSONLoader for json

- UnstructuredMarkdownLoader for md
- Etc

También se pueden cargar archivos desde diferentes fuentes que no sean el equipo host (locaciones remotas). Por ejemplo S3FileLoader que permite cargar archivos desde un Bucket de Amazon S3. El resultado de llamar un Loader es lo que llamamos Document.

Destro de un Document tenemos:

- page_content: con el contenido del archivo
- metadata: un diccionario que contiene información sobre el origen de toda esta información({"source": "facts.txt"})

Semantic Search

Understand the goal of the users search. Para esto se utilizan los Embeddings.

Un "Embedding" es una lista de números entre -1 y 1 que asigna un puntaje cuanto un pedazo de texto habla sobre una cosa o característica en particular. Es decir que tanto habla sobre la felicidad o sobre futbol, o musica, etc.

Con estas listas se puede operar. En definitiva son coordenadas de un vector. Tomar la distancia entre puntos es generar el vector de la diferencia (? esos vectores permiten realizar operaciones vectoriales.

Para saber la distancia entre 2 puntos existen dos operaciones:

- Squared L2: utiliza la distancia entre dos puntos para saber que tan similares son
- Cosine similarity: utiliza el ángulo entre 2 vectores para saber que tan similares son

Entonces la idea es:

1. Cargar el contenido de un archivo
2. Separar el contenido en fragmentos de texto llamados chunks
3. Calcular el embedding de cada chunk
4. Almacenar estos embeddings en un db especializada (Vector Store)
5. Tomar la entrada del usuario y calcular su embedding
6. Pedirle al Vector Store que me devuelva los chunks mas parecidos a la pregunta del usuario mediante Similarity/Semantic Search
7. Usar esos facts similares para generar el prompt
8. Responder la pregunta del usuario

Embeddings

Existen distintos algoritmos para calcular estos embeddings. Para esto podemos usar paquetes/liberías de Python como:

- SentenceTransformers: Se ejecuta en la pc local, y es gratis, pero consume muchos recursos. Se le puede decir que algoritmo usar, por ejemplo all-mpnet-base-v2 que genera embeddings de 768 dimensiones
- OpenAI Embeddings: Es pago, genera embeddings de 1536 dimensiones

Las embeddings generadas por distintos algoritmos no son comparables
Vector Store

- ChromaDB: Vector store que se ejecuta localmente en la pc host. Utiliza SQLite para realizar la busca por similitud entre vectores. Es open source.

RetrievalQA:

- Clase de LangChain que se encarga de levantar embeddings de la base de datos, ponerlos en un SystemMessagePromptTemplate, tomar la entrada del usuario y colocarla en un HumanMessagePromptTemplate y finalmente generar la LLMChain
- Esta pensada para una sola pregunta y una sola respuesta. No tiene concepto de memoria. No funciona muy bien en aplicaciones altamente conversacionales.

ConversationalRetrieval Chain:

- Compuesta por una memoria de mensajes y dos cadenas:
 - Condense Question Chain: Inicialmente, cuando la memoria de mensajes esta vacía, se skippea su uso.
Cuando en la memoria hay mensajes, toma la entrada del usuario mas todos los mensajes de la memoria, los pone en un prompt, la pide al LLM que usando toda esa conversación refórmele la pregunta final para que funcione de forma independiente y tenga sentido y contexto necesarios. El resultado se llama "Standalone question". Esta pregunta ya refinada se pasa a la cadena siguiente, que busca los embeddings que la respondan, arma el prompt, le pregunta al LLM y devuelve la respuesta.
 - Combine Docs Chain: funciona como una RetrievalQA, con la diferencia de que salva en la memoria, la entrada del usuario y la respuesta del LLM. No se guarda nunca la "Standalone question". Solo se salva lo que ingreso el usuario y la respuesta del LLM.

chain_type

- "stuff": Take some context from the vector store and "stuff" it into the prompt.
- "map_reduce": Build a summary of each document, then feed each summary into a final question. "System: Use the following portion of a long document to see if any of the text is relevant to answer the question. \nReturn any relevant text verbatim."
- "map_rerank": Find relevant part of each document and give it a score of how relevant it is. "Human: Use the following pieces of context to answer the question at the end. If you don't know the answer, just say that you don't know, don't try to make up an answer.\n\nIn addition to giving an answer, also return a score of how fully it answered the user's question."
- "refine": Build an initial response, then give the . Se generan 2 prompts:
 - uno para la primer pregunta
 - {
 - "prompts": [
 - "System: Context information is below.\n-----\n5. The longest word in the English language is 'pneumonoultramicroscopicsilicovolcanoconiosis.'\n6. The elephant is the only mammal that can't jump.\n-----\nGiven the context information and not prior knowledge, answer any questions\nHuman: What is an interesting fact about the English language?"
 -]
 - }
 - y otro para las siguientes:

- {
- "prompts": [
- "Human: What is an interesting fact about the English language?\nAI: An interesting fact about the English language is that the longest word in the English language is 'pneumonoultramicroscopicsilicovolcanoconiosis.'\nHuman: We have the opportunity to refine the existing answer (only if needed) with some more context below.\n-----\n1. 'Dreamt' is the only English word that ends with the letters 'mt.'\n2. An ostrich's eye is bigger than its brain.\n-----\nGiven the new context, refine the original answer to better answer the question. If the context isn't useful, return the original answer."
-]
- }

EmbeddingsRedundantFilter

- Evita tener embeddings que sean redundantes o que contengan la misma información
- Lo malo es que vuelve a calcular los embeddings.
- Para poder usarlo con un RetrievalQA es necesario crear un retriever custom. Tiene que ser una clase que extienda de BaseRetriever. Tiene que tener 2 métodos si o si para cumplir con la interfaz:
 - def get_relevant_documents(self, query):
must return a list of documents
return []
 - def aget_relevant_documents(self):
must return a list of documents
return []
- Para calcular el embedding de la pregunta del usuario podemos usar "embeddings.embed_query("holis")"
- Para hacer una búsqueda por similitud en Chroma podemos llamar a db.similarity_search(query). Va a calcular el embedding de la query y después hacer la búsqueda por similitud
- db.similarity_search_by_vector() permite hacer la búsqueda por similitud sin calcular el embedding que la pregunta del usuario.
- Chroma puede eliminar los duplicados automáticamente si utilizamos max_marginal_relevance_search_by_vector(). Recibe un valor de lambda_mult que va de 0 a 1 y que mientras mayor sea mas permeable a tener documentos similares será la búsqueda

ChatGPT Functions

En el payload que le enviamos, además de tener la lista de mensajes, podemos agregar un array de funciones. Es necesario detallar bien en la descripción de la función para que ChatGpt entienda perfectamente para que podría usar esta herramienta que le estamos ofreciendo.

Los parámetros de la función deben expresarse en JSON Schema.

La forma en que ChatGPT se comunica para usar las tools es reponiendo un mensaje con role de asistan y reemplazando el campo "content" por "function_call".

```
{
  "message": {
    "role": "assistant",
    "function_call": {
      "name": "run_query",
```

```

        "arguments": {
            "query": "SELECT COUNT(*) FROM orders;"
        }
    }
}

```

Como siempre LangChain nos permite hacer esto fácil sin tener que escribir todo el JSON a manito.

- Por defecto LangChain pone los nombres de los parámetros como “__arg1”, “__arg2”, etc.
- Para evitar eso y ponerle nombre realmente representativos que ayuden a ChatGPT a entender como usar nuestras tools, debemos hacer uso de pydantic BaseModel (v1 porque sino LangChain rompe).

Agent:

- A chain that knows how to use tools
- Will take that list of tools and convert them into JSON functions descriptions
- Still has input variables, memory, prompts, etc - all the normal things a chain has

Agent executor:

- Takes an agent and runs it until the response is NOT a function call
- Essentially a fancy while loop
- Se puede utilizar la llamada a initialize_agent que genera el executor con la referencia interna al agent
- MessagesPlaceholder(variable_name = "agent_scratchpad") -> es necesario pasar esta variable al agent para que pueda almacenar los mensajes intercambiados entre el llm y nuestra aplicación. Es básicamente como una memoria. Sirve para que el agente pueda detectar correctamente las respuestas que contengan el tipo “function_call” en lugar de “content” en la respuesta del llm. Compuesto de:
 - AI Message requesting function: AIMessageForFunction
 - Function Message: FunctionResultMessage
- MessagesPlaceholder(variable_name = "agent_scratchpad") no sirve para preservar el historial de mensajes entre el llm y el usuario.
- Los mensajes de la IA pidiendo ejecutar una función, y el resultado de las funciones son almacenados en una lista de “intermediate_steps”
- Finalmente el llm responde con un AI Message con la respuesta final para el usuario.
- El Agent Executor se ejecuta como un loop while que corre hasta que el llm llega a una respuesta final. Una vez que la ejecución termina, todos los mensajes almacenados en “intermediate_steps” son eliminados. Es por esto que el historial de mensajes entre el usuario y el llm no se preservan.
- Cuando se utiliza una memoria en un AgentExecutor, solo se preservan los HumanMessages y los AI Messages finales. Ninguna de las func request que el llm realiza ni el resultados de las mismas son preservados.
-

Tool:

- Para crear una tool simplemente se debe importar la clase Tool desde langchain.tools, y llamar a Tool.from_function(name,description,func).

- Para nombrar correctamente los parámetros, se crea una nueva clase que extienda de pydantic.v1 BaseModel, y pasarla a from_function en la variable args_schema.
- Solo sirve para funciones que reciban un solo parámetro. Si nuestra función recibe más de 1 parámetros debemos usar StructuredTool

StructuredTool:

- Es como Tool, se crea igual, recibe los mismos parámetros. La diferencia es que esta se puede usar para funciones que reciban más de 1 parámetros.

Callbacks:

- Cuando el ChatModel realiza acciones, se envían eventos (en realidad se realizan llamadas a funciones específicas que se pueden overridear si se extiende la clase correcta) que pueden ser capturados y realizar acciones.
- Podrías por ejemplo lograr cuando el ChatModel empieza, validar algo cuando el ChatModel finaliza su ejecución, etc.
- Las clases que envían estos eventos son:
 - LLM's:
 - on_llm_start()
 - on_llm_new_token()
 - on_llm_end()
 - on_llm_error()
 - Chat Models:
 - on_chat_model_start()
 - on_llm_new_token()
 - on_llm_end()
 - on_llm_error()
 - Tools:
 - on_tool_start()
 - on_tool_end
 - on_tool_error()
 - Chains:
 - on_chain_start()
 - on_chain_end()
 - on_chain_error()
 - Agents:
 - on_agent_action()
 - on_agent_finish()
- Para poder atrapar estos eventos es necesario implementar un clase handler que extienda de BaseCallbackHandler e implemente el método específico del evento a capturar. Es necesario pasar un array con las instancias de estos handlers custom a la clase que queremos escuchar en el momento de su creación. Por ejemplo:
 - chat = ChatOpenAI(
 callbacks = [handler]
)
- Pueden ser asignadas:
 - Al momento de crear la chain: callbacks=[MyCallback()] -> se asigna solo a esa instancia de la chain
 - Al momento de crear el LLM: callbacks=[MyCallback()] -> se asigna solo a esa instancia del LLM

- Cuando se llama a `__call__` de la chain: `result = chain("jdjsdj", callbacks=[MyCallback()])` -> se asigna esta callback a todo el mundo, prompt, llm, chain, subchains, todo el mundo va usar esa instancia del handler!!!
-

Uso de DB para almacenar el historial de mensajes:

- Langchain ofrece algunas herramientas para almacenar mensajes automáticamente en una gran variedad de proveedores de DB (SQL, MongoDB, DynamoDB, PostgreSQL, Cassandra, Redis, Zep, Streamlit)
- Es poco recomendado su uso
- Esto es porque al usarlas perdemos control de como se almacenan estos mensajes en la DB, que tablas hay, etc. Complica el acceso a estos mensajes en otras partes de la aplicación que no necesariamente utilizan LangChain.
- Pero en cada caso debería analizarse

Conectarse a Redis local:

`redis-cli`

`HGETALL llm_score_values`

`HGETALL llm_score_counts`

Langfuse

- Request Tracing Platform
- Para poder tener visibilidad de la información que estamos pasando en cada request y su comportamiento
- Collects data about our text generation process and presents it in a great interface
- It can be self-hosted, 100% open source y se integra muy bien con LangChain
- `prod-langfuse.fly.dev` -> versión alojada en un server del profes
-